MAX30102 Wearable Bio-Sensor Demo 1 Integration Guide

Raspberry Pi Zero W & MAX30102 Step-by-Step Setup & Verification Guide

Step 1: Raspberry Pi Pin Setup & Header Soldering

Solder header pins onto the following 4 physical pins on your Raspberry Pi Zero W header:

RPi Physical Pin	Pin Name	Connected Destination	Function
Pin 1	3.3V Power	Resistor Divider Top & MAX30102 Pins 9 & 10 ($V_{\text{LED+}}$)	Power LEDs & 1.8V d
Pin 3	GPIO 2 (SDA)	MAX30102 Pin 3 (SDA)	I^2C Data Line
Pin 5	GPIO 3 (SCL)	MAX30102 Pin 2 (SCL)	I^2C Clock Line
Pin 6	Ground (GND)	Resistor Divider Bottom & MAX30102 Pins 4 & 12	Common System Grou

Step 2: Breadboard Resistor Divider Circuit (V_{DD} Power Supply)

The MAX30102 digital core requires an operating voltage (V_{DD}) between 1.7V and 2.0V[cite: 2] (absolute maximum rating: 2.2V[cite: 2]). Voltages below 1.7V trigger Undervoltage Lockout (UVLO) and hold the device in reset[cite: 2].

- **Top Side (94 Ω)**: Connect two 47 Ω resistors in series end-to-end from RPi Pin 1 (3.3V) to an empty breadboard row (the **1.8V Node**).
- **Bottom Side (100 Ω)**: Connect one 100 Ω resistor from the **1.8V Node** to RPi Pin 6 (GND).

$$V_{\text{out}} = 3.3\text{V} \times \frac{100\,\Omega}{47\,\Omega + 100\,\Omega} \approx 2.2\text{V}$$

Step 3: MAX30102 Sensor Wiring & Package Alignment

The MAX30102 bottom pads feature staggered pin offsets across the chip body:

- **Pin 12 (GND)** sits directly across from **Pin 3 (SDA)**.
- **Pin 4 (PGND)** sits directly across from **Pin 11 (V_{DD})**.

MAX30102 Pin	Pin Function	Breadboard / Raspberry Pi Destination
Pin 11	V_{DD} (Analog Core Power)	1.8V Node (Resistor Divider Center)
Pins 9 & 10	$V_{\text{LED+}}$ (LED Supply)	3.3V Node (RPi Pin 1)
Pins 4 & 12	PGND & GND (Grounds)	GND Node (RPi Pin 6)
Pin 3	SDA (I^2C Data)	RPi Pin 3 (GPIO 2 / SDA)
Pin 2	SCL (I^2C Clock)	RPi Pin 5 (GPIO 3 / SCL)

Step 4: Raspberry Pi OS Software Configuration

1. **Enable I^2C Bus**: Run `sudo raspi-config` → Interface Options → I2C → Select "Yes" → Finish.
2. **Install Required Packages**:

   ```
   sudo apt update && sudo apt install -y i2c-tools python3-smbus python3-pip
   pip3 install smbus2
   ```

3. **Verify Device Node**: Run `ls -l /dev/i2c*` to confirm `/dev/i2c-1` is active.

Step 5: Hardware I^{2}C Verification

Run the bus scan tool to check hardware communication:

```
i2cdetect -y 1
```

Expected Output: Address **0x57** (shown as `57`) appears at row `50:` under column `7`. This confirms the MAX30102 is powered, out of reset, and acknowledging commands.

Step 6: Python Demo Code (`max30102_demo.py`)

Save and run the following Python script to verify Part ID (`0x15`), read die temperature, configure SpO_2 mode, and stream live 18-bit PPG data:

```
import time
import smbus2

MAX30102_ADDR      = 0x57  # 7-bit I2C Slave Address
REG_FIFO_WR_PTR    = 0x04
REG_OVF_COUNTER    = 0x05
REG_FIFO_RD_PTR    = 0x06
REG_FIFO_DATA      = 0x07
REG_FIFO_CONFIG    = 0x08
REG_MODE_CONFIG    = 0x09
REG_SPO2_CONFIG    = 0x0A
REG_LED1_PA        = 0x0C  # RED LED
REG_LED2_PA        = 0x0D  # IR LED
REG_TEMP_INT       = 0x1F
REG_TEMP_FRAC      = 0x20
REG_TEMP_CONFIG    = 0x21
REG_PART_ID        = 0xFF  # Expected Value: 0x15

class MAX30102:
    def __init__(self, bus_number=1):
        self.bus = smbus2.SMBus(bus_number)
        self.verify_part_id()
        self.reset()
        self.setup()

    def verify_part_id(self):
        part_id = self.bus.read_byte_data(MAX30102_ADDR, REG_PART_ID)
        if part_id != 0x15:
            raise RuntimeError(f"Unexpected Part ID: 0x{part_id:02X} (Expected 0x15)")
        print("MAX30102 detected successfully.")

    def reset(self):
        self.bus.write_byte_data(MAX30102_ADDR, REG_MODE_CONFIG, 0x40)
        time.sleep(0.1)

    def setup(self):
        # FIFO Config: Sample Averaging = 4, Rollover = On, Almost Full = 15
        self.bus.write_byte_data(MAX30102_ADDR, REG_FIFO_CONFIG, 0x5F)
        # SpO2 Config: Range = 4096nA, Sample Rate = 100Hz, Pulse Width = 411us (18-
            bit)
        self.bus.write_byte_data(MAX30102_ADDR, REG_SPO2_CONFIG, 0x27)
        # LED Pulse Amplitudes (~7.2mA)
        self.bus.write_byte_data(MAX30102_ADDR, REG_LED1_PA, 0x24)
        self.bus.write_byte_data(MAX30102_ADDR, REG_LED2_PA, 0x24)
        # Mode Config: SpO2 Mode (Red + IR Active)
        self.bus.write_byte_data(MAX30102_ADDR, REG_MODE_CONFIG, 0x03)
        # Clear FIFO Pointers
        self.bus.write_byte_data(MAX30102_ADDR, REG_FIFO_WR_PTR, 0x00)
        self.bus.write_byte_data(MAX30102_ADDR, REG_OVF_COUNTER, 0x00)
        self.bus.write_byte_data(MAX30102_ADDR, REG_FIFO_RD_PTR, 0x00)

    def read_fifo(self):
```

```
        wr_ptr = self.bus.read_byte_data(MAX30102_ADDR, REG_FIFO_WR_PTR)
        rd_ptr = self.bus.read_byte_data(MAX30102_ADDR, REG_FIFO_RD_PTR)
        if wr_ptr == rd_ptr:
            return []
        num_samples = (wr_ptr - rd_ptr) % 32
        samples = []
        for _ in range(num_samples):
            raw_data = self.bus.read_i2c_block_data(MAX30102_ADDR, REG_FIFO_DATA, 6)
            red_raw = ((raw_data[0] << 16) | (raw_data[1] << 8) | raw_data[2]) & 0
                x03FFFF
            ir_raw  = ((raw_data[3] << 16) | (raw_data[4] << 8) | raw_data[5]) & 0
                x03FFFF
            samples.append((red_raw, ir_raw))
        return samples

    def read_temperature(self):
        self.bus.write_byte_data(MAX30102_ADDR, REG_TEMP_CONFIG, 0x01)
        time.sleep(0.03)
        t_int = self.bus.read_byte_data(MAX30102_ADDR, REG_TEMP_INT)
        t_frac = self.bus.read_byte_data(MAX30102_ADDR, REG_TEMP_FRAC)
        if t_int > 127:
            t_int -= 256
        return t_int + (t_frac * 0.0625)

if __name__ == "__main__":
    sensor = MAX30102()
    print("Reading bio-sensor data... Press Ctrl+C to stop.")
    try:
        temp = sensor.read_temperature()
        print(f"Die Temperature: {temp:.2f} deg C\n")
        while True:
            samples = sensor.read_fifo()
            for red, ir in samples:
                print(f"RED: {red:6d} | IR: {ir:6d}")
            time.sleep(0.05)
    except KeyboardInterrupt:
        print("\nDemo stopped.")
```

Step 7: Team Technical Briefing & Demo 2 Roadmap

- **Passive Resistor Divider Limitations**: The resistor divider works for Demo 1, but because the MAX30102's current consumption shifts dynamically during optical sampling (0.7 μA standby to 1.2 mA active)[cite: 2], V_{DD} can sag under heavy load[cite: 2].
- **Upgrade Plan for Demo 2 & Multi-Sensor Breakout**: Replace the resistor divider with a dedicated **1.8V Low-Dropout (LDO) Linear Regulator** (such as the `MCP1700-1802E`) and 0.1 μF / 4.7 μF decoupling capacitors to ensure long-term power stability as additional breakout sensors are integrated into the wearable system.